

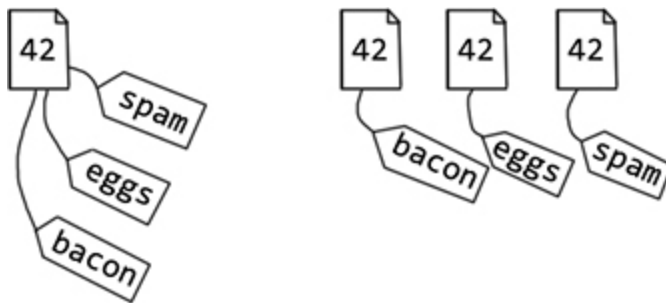
separate places in the computer's memory. You can confirm this by checking their different IDs using the `id()` function:

---

```
>>> a = 42
>>> b = 42.0
>>> a == b
True
>>> a is b
False
>>> id(a), id(b)
(140718571382896, 2526629638888)
```

---

When Python creates a new integer object and stores it in memory, that object creation takes very little time. As a tiny optimization, CPython (the Python interpreter available for download at <https://python.org>) creates integer objects for `-5` to `256` at the start of every program. These integers are called *preallocated integers*, and CPython automatically creates objects for them because they're fairly common: a program is more likely to use the integer `0` or `2` than, say, `1729`. When creating a new integer object in memory, CPython first checks whether it's between `-5` and `256`. If so, CPython saves time by simply returning the existing integer object instead of creating a new one. This behavior also saves memory by not storing duplicate small integers, as illustrated in [Figure 9-1](#).



[Figure 9-1](#): Python saves memory by using multiple references to a single integer object (left) instead of separate, duplicate integer objects for each reference (right).

Because of this optimization, certain contrived situations can produce bizarre results. To see an example of one, enter the following into the interactive shell: